

ranges::iota, ranges::shift_left, and ranges::shift_right

Document #: P2440R0
Date: 2021-09-12
Project: Programming Language C++
Audience: LEWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

Contents

- [1 Abstract](#)
- [2 Discussion](#)
 - [2.1 ranges::iota](#)
 - [2.2 ranges::shift_left and ranges::shift_right](#)
- [3 Wording](#)
 - [3.1 ranges::iota](#)
 - [3.2 ranges::shift_left and ranges::shift_right](#)
- [4 References](#)

§ 1 **Abstract**

This paper proposes adding the algorithms `ranges::iota`, `ranges::shift_left`, and `ranges::shift_right`, to match their `std` counterparts.

§ 2 **Discussion**

§ 2.1 **ranges::iota**

As [\[P2214R0\]](#) explains, while conceptification of other numeric algorithms is a far more complex endeavor (see, e.g., [\[P1813R0\]](#)), `iota` is straightforward:

We already have `views::iota` in C++20, which importantly means that we already have all the correct constraints in place. In that sense, it almost takes less time to adopt `ranges::iota` than it would take to discuss whether or not it's worth spending time adopting it.

The `ranges::iota` algorithm isn't redundant to `views::iota` either: the algorithm determines the number of values to write based on the output range, while using `ranges::copy` or similar algorithms with `views::iota` requires that information to be computed upfront. When you already have a range, `ranges::iota` can be more convenient and efficient.

Following the law of useful returns, this paper proposes returning both the end of the range and the final value.

§ 2.2 `ranges::shift_left` and `ranges::shift_right`

These were proposed in [P1243R3] but removed in Prague due to concerns about the return type of `shift_left` losing information about the original end of the range. There were particularly concerns about difficulty in recovering the original end if sentinels are involved, since the elements between the end of the new range and the previous end could have been moved from.

However, this argument overlooks the fact that:

- when the returned range is not empty, we can simply take its `end()` and advance it by the shift amount `n` to obtain the original end. This is a potentially linear time operation, but it is not impossible to recover the information.
- when the returned range is empty (that is, the shift amount is greater than the size of the range), simply adding `n` is not possible, but in this case the algorithm may not necessarily have computed the end iterator of the original range: all it needs is the size, which can be computed from `last - first` if their types model `sized_sentinel_for`, or from the range if its type models `sized_range`. Additionally, we guarantee that in this case the range is unchanged, so the sentinel remains usable.

Moreover, the standard library already provides a view factory for users who want to iterate `[new_last, last): views::counted`. In some cases, using `counted_iterator` might even be more efficient by reducing iterator comparisons to integer comparisons (comparing `counted_iterators` only need to compare the counter).

Meanwhile, just returning a subrange that represents the new range of elements instead of a more complex, bespoke type that users need to decompose themselves is substantially more convenient, and it is hardly unprecedented for some algorithms to not return the end of the range despite necessarily computing it (e.g., `ranges::count`, `ranges::min/max/minmax_element`), and here we don't even necessarily compute it. It also allows `shift_left` and `shift_right` to have the same return type, without having to complicate the latter.

This paper therefore proposes maintaining the [P1243R3] design: `ranges::shift_left` and `ranges::shift_right` should return a subrange that represents the new range after the shift.

§ 3 Wording

§ 3.1 `ranges::iota`

1. Edit 25.9 [numeric.ops.overview], header `<numeric>` synopsis, as indicated:

```
namespace std {  
+ namespace ranges {  
+   // 25.5 [algorithms.results], algorithm result types  
+ }
```

```

+   template<class O, class T>
+       struct out_value_result;
+   }

    // [...]

    // 25.10.13 \[numeric.iota\], iota
    template<class ForwardIterator, class T>
        constexpr void iota(ForwardIterator first, ForwardIterator last, T
            value);

+   namespace ranges {
+       template<class O, class T>
+           using iota_result = out_value_result<O, T>;
+       +
+       template<input_or_output_iterator O, sentinel_for<O> S,
+           weakly_incrementable T>
+           requires indirectly_writable<O, const T>
+           constexpr iota_result<O, T> iota(O first, S last, T value);
+       +
+       template<weakly_incrementable T, output_range<const T> R>
+           constexpr iota_result<borrowed_iterator_t<R>, T> iota(R&& r, T
+               value);
+   }

    // [...]
}

```

2. Add the following to 25.5 [\[algorithms.results\]](#):

```

namespace std::ranges {
    // ...

    template<class O, class T>
    struct out_value_result {
        [[no_unique_address]] O out;
        [[no_unique_address]] T value;

        template<class O2, class T2>
            requires convertible_to<const O&, O2> && convertible_to<const T&, T2>
            constexpr operator out_value_result<T2, O2>() const & {
                return {out, value};
            }

        template<class O2, class T2>
            requires convertible_to<O, O2> && convertible_to<T, T2>
            constexpr operator out_value_result<T2, O2>() && {
                return {std::move(out), std::move(value)};
            }
    };
}

```

3. Add the following to 25.10.13 [\[numeric.iota\]](#):

```

template<input_or_output_iterator O, sentinel_for<O> S, weakly_incrementable
    T>
    requires indirectly_writable<O, const T>

```

```
constexpr ranges::iota_result<0, T> ranges::iota(0 first, S last, T
    value);

template<weakly_incrementable T, output_range<const T> R>
constexpr ranges::iota_result<borrowed_iterator_t<R>, T> ranges::iota(R&&
    r, T value);

? Effects: Equivalent to: [ Drafting note: Matching range-v3, we write
    as_const(value) to the output to enforce that value should not be modified by
    the writing operation. ]

    while (first != last) {
        *first = as_const(value);
        ++first;
        ++value;
    }

    return {std::move(first), std::move(value)};
```

4. Add the following macro definition to 17.3.2 [\[version.syn\]](#), header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this proposal:

```
#define __cpp_lib_ranges_iota 20XXXXL // also in <numeric>
```

§ 3.2 `ranges::shift_left` and `ranges::shift_right`

1. Edit 25.4 [\[algorithm.syn\]](#), header `<algorithm>` synopsis, as indicated:

```
namespace std {

    // [...]

    // 25.7.14 \[alg.shift\], shift
    template<class ForwardIterator>
        constexpr ForwardIterator
            shift_left(ForwardIterator first, ForwardIterator last,
                      typename iterator_traits<ForwardIterator>::difference_type
                        n);
    template<class ExecutionPolicy, class ForwardIterator>
        ForwardIterator
            shift_left(ExecutionPolicy&& exec, // see
                      [algorithms.parallel.overloads]
                      ForwardIterator first, ForwardIterator last,
                      typename iterator_traits<ForwardIterator>::difference_type
                        n);

+ namespace ranges {
+     template<permutable I, sentinel_for<I> S>
+         constexpr subrange<I> shift_left(I first, S last,
+           iter_difference_t<I> n);
+     template<forward_range R>
+         requires permutable<iterator_t<R>>
+         constexpr borrowed_subrange_t<R> shift_left(R&& r,
+           range_difference_t<R> n);
```

```

+ }

template<class ForwardIterator>
constexpr ForwardIterator
    shift_right(ForwardIterator first, ForwardIterator last,
                typename
                    iterator_traits<ForwardIterator>::difference_type n);
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator
    shift_right(ExecutionPolicy&& exec, // see
                [algorithms.parallel.overloads]
                ForwardIterator first, ForwardIterator last,
                typename
                    iterator_traits<ForwardIterator>::difference_type n);

+ namespace ranges {
+     template<permutable I, sentinel_for<I> S>
+         constexpr subrange<I> shift_right(I first, S last,
+             iter_difference_t<I> n);
+     template<forward_range R>
+         requires permutable<iterator_t<R>>
+         constexpr borrowed_subrange_t<R> shift_right(R&& r,
+             range_difference_t<R> n);
+ }

    // [...]
}

```

2. Edit 25.7.14 [\[alg.shift\]](#) as indicated:

```

template<class ForwardIterator>
constexpr ForwardIterator
    shift_left(ForwardIterator first, ForwardIterator last,
               typename
                   iterator_traits<ForwardIterator>::difference_type n);
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator
    shift_left(ExecutionPolicy&& exec, ForwardIterator first,
               ForwardIterator last,
               typename
                   iterator_traits<ForwardIterator>::difference_type n);
+
+ template<permutable I, sentinel_for<I> S>
+     constexpr subrange<I> ranges::shift_left(I first, S last,
+         iter_difference_t<I> n);
+ template<forward_range R>
+     requires permutable<iterator_t<R>>
+     constexpr borrowed_subrange_t<R> ranges::shift_left(R&& r,
+         range_difference_t<R> n);

```

- 1 *Preconditions:* $n \geq 0$ is true. For the overloads in namespace `std`, the type of `*first` meets the *Cpp17MoveAssignable* requirements.
- 2 *Effects:* If $n == 0$ or $n \geq \text{last} - \text{first}$, does nothing. Otherwise, moves the element from position `first + n + i` into position `first + i` for each non-negative integer $i < (\text{last} - \text{first}) - n$. For the overloads without an

ExecutionPolicy template parameter. In the first overload case, does so in order starting from $i = 0$ and proceeding to $i = (\text{last} - \text{first}) - n - 1$.

- 3 Returns: Let *NEW_LAST* be $\text{first} + (\text{last} - \text{first} - n)$ if $n < \text{last} - \text{first}$, otherwise *first*.

(3.1) — *NEW_LAST* for the overloads in namespace *std*.

(3.2) — $\{\text{first}, \text{NEW_LAST}\}$ for the overloads in namespace *ranges*.

- 4 Complexity: At most $(\text{last} - \text{first}) - n$ assignments.

```
template<class ForwardIterator>
constexpr ForwardIterator
    shift_right(ForwardIterator first, ForwardIterator last,
                typename
                    iterator_traits<ForwardIterator>::difference_type n);
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator
    shift_right(ExecutionPolicy&& exec, ForwardIterator first,
                ForwardIterator last,
                typename
                    iterator_traits<ForwardIterator>::difference_type n);
+
+template<permutable I, sentinel_for<I> S>
+ constexpr subrange<I> ranges::shift_right(I first, S last,
+     iter_difference_t<I> n);
+template<forward_range R>
+ requires permutable<iterator_t<R>>
+ constexpr borrowed_subrange_t<R> ranges::shift_right(R&& r,
+     range_difference_t<R> n);
```

- 5 Preconditions: $n \geq 0$ is true. For the overloads in namespace *std*, *first* must be a pointer to a type that meets the *Cpp17MoveAssignable* requirements, and *first* must be a *Cpp17BidirectionalIterator* or the *Cpp17ValueSwappable* requirements.

- 6 Effects: If $n == 0$ or $n \geq \text{last} - \text{first}$, does nothing. Otherwise, moves the element from position $\text{first} + i$ into position $\text{first} + n + i$ for each non-negative integer $i < (\text{last} - \text{first}) - n$. In the first overload case, if *first* meets the *Cpp17BidirectionalIterator* requirements, does so in order starting from $i = (\text{last} - \text{first}) - n - 1$ and proceeding to $i = 0$ if.

(6.1) — for the overload in namespace *std* without an ExecutionPolicy template parameter, *ForwardIterator* meets the *Cpp17BidirectionalIterator* requirements.

(6.2) — for the overloads in namespace *ranges*, $\text{decltype}(\text{first})$ models *bidirectional_iterator*.

- 7 Returns: Let *NEW_FIRST* be $\text{first} + n$ if $n < \text{last} - \text{first}$, otherwise *last*.

- (7.1) — *NEW_FIRST* for the overloads in namespace `std`.
 - (7.2) — `{NEW_FIRST, last}` for the overloads in namespace `ranges`.
- 8 *Complexity*: At most `(last - first) - n` assignments or swaps.

3. Update the value of `__cpp_lib_shift` in 17.3.2 [\[version.syn\]](#) to reflect the date of adoption of this proposal.

§ 4 References

[P1243R3] Dan Raviv. 2020-01-09. Rangify New Algorithms.

<https://wg21.link/p1243r3>

[P1813R0] Christopher Di Bella. 2019-08-02. A Concept Design for the Numeric Algorithms.

<https://wg21.link/p1813r0>

[P2214R0] Barry Revzin, Conor Hoekstra, Tim Song. 2020-10-15. A Plan for C++23 Ranges.

<https://wg21.link/p2214r0>